Group Project 2: Domotics

Operating Systems

Professors:
Domenico Siracusa
Matteo Franzil

University of **Trento**

September 2026

Group 22:
Mattia Biral — 252925
Leonardo Paiola — 252756
Elisa Potrich — 250782

Report

Contents

Device Implementation

Leaf Devices

Bulbs, Windows and Fridges use the main thread to receive command requests, execute them and send responses after having waited a random time between 1 and 3 seconds.

When they receive a switch command they modify their state accordingly and update their `last_turned_on` and `last_turned_off`, or `last_opened` and `last_closed`, timestamps, used to calculate the last on/open time (not cumulative total, as specified by professor Franzil).

This time is always transmitted and displayed in minutes for Bulbs and Windows, and in seconds for Fridges, as their automatic close action limits the last open time.

Any signal between `SIGTERM`, `SIGINT` and `SIGPIPE` cause the process to exit and report an unexpected shutdown.

Fridge

Additionally the Fridge has, as mentioned, an automatic close action, which is performed through a detached thread.

The thread is created when the door is opened, it sleeps `autoclose_delay` seconds and then closes the door, if the door in closed by an explicit request instead, the thread is cancelled.

It's a detached thread so that it doesn't need to be joined for its resources to be freed when it completes, because it would be difficult to determine when and where it should be joined.

The Fridge temperature is calculated through a linear approximation, the temperature oscillates around the `thermostat` if the door is closed, when it is opened it starts rising towards ambient temperature, and lowers again when the door is closed.

This calculation is performed every time the temperature is needed, there is no thread dedicated to periodically updating it, making it lighter.

A mutex is used to ensure data is accessed and modified by only one thread at a time preventing inconsistencies.

The termostat and the fill percentage can only be modified through the Manual Interaction Program, and the Controller does not recognize the corresponding commands as valid.

Control Devices

As opposed to leaf devices the `SIGPIPE` signal is ignored, to prevent a domino effect, otherwise a children crash followed by a request to be forwarded to it (before it is informed to remove it from its children) would also cause it to crash, a write error still occurs and is notified to the user.

All control devices can manage a routing table, which contains information about all their direct and indirect children, to allow requests to specific devices to be forwarded.

Hubs and Timers

We decided to allow devices to be added to Hubs only if they match the type of the other children or of the parent, this way an Hub performs the same action on all children if it's the destination of a switch command.

Furthermore, for an info request the collected states of the children are of the same type, power on/off or open/close.

This also allows the Timer to know exactly what type of action it has to perform on its child at `begin` and `end`.

So for example a Hub with child type Bulb, can only have as children one or more of the following:

- Bulbs;
- Hubs with child type Bulb (or empty);
- Timers with child type Bulb (or empty).

Hubs and Timers do not cache state information about their children, instead they insert a pending response in their list, when they receive a response they search in the list and modify data accordingly, when a pending response receives all expected responses it becomes complete and is sent upwards to the parent.

For an info command they respond with the state, which is undefined if they have no children or manual override if the states of the children are different, and the maximum last time on/open; other information received by their children is discarded as the user can still issue an individual info command directly to them.

The main thread is dedicated to top-down message flow and it receives requests. If the destination is the device itself, then it responds directly to its parent, otherwise it forwards the request to the correct child, if possible.

Another joinable thread is used to handle bottom-up communication: it receives responses from its children, if the response was pending it is handled as described above, otherwise it is simply forwarded to the parent.

On exit, successful or not, all threads except the exiting one can be blocked on a read call (which is a cancellation point), so they are cancelled and then joined.

The random delay is only added once, in the top-down thread, and not in the bottom-up one, otherwise the total delay would become too great and the user would have to wait times close to a minute if the branch is only a few nodes deep.

Timer

A Timer additionally includes its `begin` and `end` to its info response.

These registries contain the time at which an automatic on/open or off/closed action is triggered, stored as the number of minutes from midnight (e.g. 2:05 AM would be 125).

A joinable thread is used to handle such automatic actions, it sleeps the amount of time remaining to the next `begin` or `end` in a loop, it is cancelled, joined and recreated only if one of such registries changes.

Controller

The controller also exports its routing table to a `ipc/devices.registry` file, to allow the Manual Interaction to know the type of the target device and check if the user command is compatible with it (e.g. a switch power command will cause a `DEVICE_TYPE_MISMATCH` error if the target is not a Bulb, or a control device with Bulb children).

This export is done first to a temporary file, which is then renamed to replace the old registry file, preventing possible access conflicts and errors.

The main thread is dedicated to parsing, checking and executing user commands, another joinable thread reads responses from its children and sometimes uses them to update its routing information and send additional commands to the devices.

The Controller has a signal handler for `SIGCHLD`, it detects if a device has exited with errors, removes it from routing information, performs pipe cleanup and notifies the parent.

These actions need access to data shared between threads, so a mutex is used to ensure data remains consistent, however since it might be already locked by the main thread (which is interrupted to execute the handler until it finishes), to prevent a deadlock the signal handler only creates a detached thread dedicated to this cleanup which then locks the mutex.

Device Creation

Each device type is implemented in a different executable, this allows the use of global variables to share data between threads, and it also means that each process has in memory only the code of its specific device type.

To add a new device, the Controller forks and then calls `exec` which will replace the process memory image with the one of the wanted executable, also destroying all threads and signal handlers that may have been created in the Controller, that are surely unwanted in the new device.

Also files opened in the Controller with the flag `O_CLOEXEC` are automatically closed.

Messages

Devices communicate with the use of named pipes and formatted messages.

Format

Requests and responses are formatted as strings containing positive integer numbers separated by spaces, this allows easier debugging since the messages can be easily printed and are more human readable, also functions dedicated to number parsing can be reused.

To allow the devices to access data rapidly and easily when a message is received, it is first parsed and data is put in structs.

When a device needs to send a message instead, it puts the information in a struct which is then formatted to a string.

The POSIX standard guarantees that concurrent writes are atomic if the size is less than `PIPE_BUF`, which is required to be at least 512 bytes (on most Linux systems it's now 4096 bytes).

Since the formatted messages fit in 64 bytes, and every write uses exactly that size, multiple processes can write to the same pipe without the messages getting interleaved.

Additionally, writes and corresponding reads always use the maximum message size, this guarantees that when a read is performed only the next message is actually read, avoiding complex read logics.

Pipes

Named pipes are contained in the `ipc/` folder, each leaf device communicates using two pipes that can be visualized in Figure 1:

- `"<device_id>_down.fifo"`, where it receives requests from its parent;
- `"<parent_id>_up.fifo"`, where it sends responses to its parent.

Each Hub or Timer additionally has a pipe `"<device_id>_up.fifo"` where it receives responses from its children as illustrated in Figure 2, and opens the pipes of all its children, one for the Timer, to be able to send them requests.

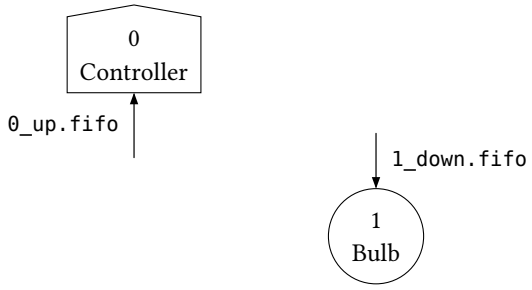

Figure 1: leaf device named pipes

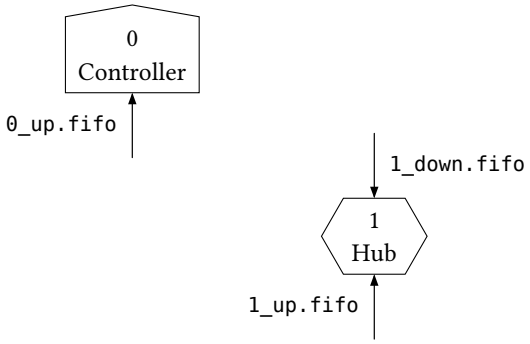

Figure 2: control device named pipes

A control device opens its "up" pipe, where it receives responses, in read-write mode, this way the `open` doesn't block and the `read` doesn't return `0` (end of file) when all children are removed but it keeps waiting for someone to write.

Routing Data

A routing table is implemented as a hashmap that contains information about every device that is currently a direct or indirect child.

Each entry contains the ID of the device and of its parent, the device type and a `next_hop_fd`, the file descriptor where the message should be forwarded for it to arrive at its final destination.

The table is organized as an array of buckets, each bucket is a linked list of entries ordered by ID, this improve search time for forwarding messages.

Link change parent responses also contain the device type of the added child, or the new device type for link remove child responses, to allow its parents to update their device type accordingly.

Link

A link command involves a device that is moved from one point of the topology to another.

When a user issues a link command, a link change parent message is sent to the device to be moved, and a successful response is directly sent to its new parent.

The new parent, and its parents, can extrapolate information from such response, knowing that there is a new direct or indirect child in the branch.

When a control device is moved, it sends a list of change parent messages, so that its parents can also learn about all its direct and indirect children, so that requests to them can be correctly forwarded.

When the Controller receives a successful link change parent response, or it detects a device crash, it sends a link remove child request to the old parent, so that it removes it from its routing information, from its pending responses and eventually closes the pipe if it was a direct children.

User Interface

Since the Controller can simultaneously receive commands from the user and responses from the devices, to avoid the prints to graphically interleave with the user input the terminal is divided in two areas with the help of the `ncurses` library:

- the top two thirds of the terminal are dedicated to output and display error messages and device responses;
- the bottom third is dedicated to user input and also displays the last user commands.

In extreme cases devices that share the same output streams of the Controller can directly print to `stderr`.

To avoid such prints to mess with the `ncurses` interface, `stderr` is redirected to a named pipe, the Controller then reads the pipe, using another joinable thread, and prints the messages using the `ncurses` library.

It is also possible to use a normal terminal interface by passing `--no-ncurses` as argument to the Controller.

Errors

Error Handling

Errors are categorized by cause, a function is used to print as much information as available about the specific error, where it happened, and the `errno` if caused by a system call.

Functions that can fail return an error code, if a function returns a numeric value used for other purposes, the error code is returned as a negative number.

When an error is encountered:
- if possible, a response containing the error code is sent to the parent;
- if not, it is directly printed to `stderr`.

If such error does not allow the device to continue working properly, e.g. the mutex could not be unlocked, the device then enters its shutdown procedure and exits with the corresponding error code.

Manual Interaction

The `manual_interaction` executable parses the user command, the first argument is always the target device ID, and it exposes the following interface:
- `switch <power/open/close> <on/off>` which allows to change the state of a device through its switches;
- `set <begin/end/delay/thermostat/perc> <value>` which allows to edit device registries;
- `info`.

It writes directly to the named pipe where the target device usually receives commands coming from the Controller.

The response is sent by the device to its parent and will eventually reach the Controller which will display it to the user.

Scenario

The `commands.scenario` file contains a list of commands that are executed by the Controller before any other user command.

It builds a topology on which the different commands can be tested, the user can send switch and info commands to any device, or set registry commands to the specific device which registry has to be modified.

The exit command can then be used to delete all devices and shutdown the Controller.